AUCA BIG DATA ANALYTICS FINAL PROJECT: Distributed Multi-Model Analytics for E-commerce Data

Oct 28, 2026

PROJECT OVERVIEW

This final project challenges you to design and implement a multi-faceted analytics system for large-scale e-commerce data. You will leverage the distinct strengths of MongoDB (document model), HBase (wide-column model), and Apache Spark (distributed processing) to perform a range of analytical tasks. This project will allow you to demonstrate your understanding of selecting appropriate data models and processing engines based on specific analytical requirements and data characteristics.

- **Project Format:** Individual Project
- **Due Date:**11:59am, 11-11-2026
- **Focus:** Strategic application of MongoDB, HBase, and Apache Spark for big data analytics in an e-commerce context.

KEY LEARNING OBJECTIVES

- Understand and articulate the trade-offs between different NoSQL data models (document vs. wide-column).
- Design efficient data schemas for MongoDB and HBase tailored to specific query patterns.
- Implement data ingestion and querying techniques for MongoDB and HBase.
- Develop batch and (simulated) stream processing applications using Apache Spark.
- Integrate data from multiple diverse data stores to perform complex analytics.
- Generate meaningful business insights from large datasets and present them effectively.

DATASET

You will use the provided Python script (dataset_generator.py - see the accompanying code file) to generate the synthetic e-commerce dataset. You will need to install the Faker library (pip install faker pandas) to run the script.

E-Commerce Analytics Dataset Description

This dataset simulates a comprehensive e-commerce platform's data ecosystem, providing rich, interconnected data specifically designed to demonstrate the strengths of multiple database models for big data analytics. The dataset consists of users, products, categories, sessions, and transactions spanning 90 days of activity.

Data Entities and Relationships

- **Users (users.json)**
  - User profiles with demographic and registration information.
  - *Sample Data:*


```
{
  "user_id": "user_000042",
  "geo_data": {
    "city": "North Michaelville",
    "state": "WY",
    "country": "US"
  },
  "registration_date": "2024-12-15T08:42:13",
  "last_active": "2025-03-12T16:23:47"
}
{
  "user_id": "user_000173",
  "geo_data": {
    "city": "East Angela",
    "state": "MN",
    "country": "US"
  },
  "registration_date": "2024-11-30T14:27:56",
  "last_active": "2025-04-02T09:15:32"
}
```


- **Categories (categories.json)**
  - Hierarchical product classification system.
  - *Sample Data:*


```
{
  "category_id": "cat_007",
  "name": "Johnson-Williams",
  "subcategories": [
    {
```

```
      "subcategory_id": "sub_007_00",
      "name": "Synergize Scalable E-Markets",
      "profit_margin": 0.27
    },
    {
      "subcategory_id": "sub_007_01",
      "name": "Mesh Virtual Deliverables",
      "profit_margin": 0.18
    }
  ]
}
```


- **Products (products.json)**
  - Product catalog with inventory information and price history.
  - *Sample Data:*


```
{
  "product_id": "prod_00123",
  "name": "Innovative Executive Paradigm",
  "category_id": "cat_007",
  "subcategory_id": "sub_007_01",
  "base_price": 129.99,
  "current_stock": 47,
  "is_active": true,
  "price_history": [
    {
      "price": 149.99,
      "date": "2024-12-20T00:00:00"
    },
    {
      "price": 129.99,
      "date": "2025-02-15T00:00:00"
    }
  ],
  "creation_date": "2024-12-20T00:00:00"
```

```
}

{
  "product_id": "prod_03267",
  "name": "Ergonomic Concrete Interface",
  "category_id": "cat_018",
  "subcategory_id": "sub_018_03",
  "base_price": 84.50,
  "current_stock": 0,
  "is_active": false,
  "price_history": [
    {
      "price": 74.99,
      "date": "2025-01-05T00:00:00"
    },
    {
      "price": 84.50,
      "date": "2025-02-28T00:00:00"
    }
  ],
  "creation_date": "2025-01-05T00:00:00"
}
```

- **Sessions (sessions_0.json, sessions_1.json, etc.)**
  - Detailed browsing activity for users, divided into multiple files due to volume.
  - *Sample Data:*

```
{
  "session_id": "sess_a7b3c9d8e2",
  "user_id": "user_000042",
  "start_time": "2025-03-12T14:37:22",
  "end_time": "2025-03-12T14:52:41",
  "duration_seconds": 919,
  "geo_data": {
    "city": "North Michaelville",
```

```
  "state": "WY",
  "country": "US",
  "ip_address": "172.24.78.193"
},
"device_profile": {
  "type": "mobile",
  "os": "iOS",
  "browser": "Safari"
},
"viewed_products": ["prod_00123", "prod_02456", "prod_01872"],
"page_views": [
  {
    "timestamp": "2025-03-12T14:37:22",
    "page_type": "home",
    "product_id": null,
    "category_id": null,
    "view_duration": 42
  },
  {
    "timestamp": "2025-03-12T14:38:04",
    "page_type": "category_listing",
    "product_id": null,
    "category_id": "cat_007",
    "view_duration": 67
  },
  {
    "timestamp": "2025-03-12T14:39:11",
    "page_type": "product_detail",
    "product_id": "prod_00123",
    "category_id": "cat_007",
    "view_duration": 124
  },
  {
    "timestamp": "2025-03-12T14:41:15",
    "page_type": "cart",
    "product_id": null,
    "category_id": null,
```

```
      "view_duration": 56
    }
  ],
  "cart_contents": {
    "prod_00123": {
      "quantity": 2,
      "price": 129.99
    }
  },
  "conversion_status": "converted",
  "referrer": "search_engine"
}
```

- **Transactions (transactions.json)**
  - Purchase records with line items, payment details, and session linkage.
  - *Sample Data:*

```
{
  "transaction_id": "txn_c8d9e7f3a2b1",
  "session_id": "sess_a7b3c9d8e2",
  "user_id": "user_000042",
  "timestamp": "2025-03-12T14:52:41",
  "items": [
    {
      "product_id": "prod_00123",
      "quantity": 2,
      "unit_price": 129.99,
      "subtotal": 259.98
    }
  ],
  "subtotal": 259.98,
  "discount": 25.99,
  "total": 233.99,
  "payment_method": "credit_card",
  "status": "completed"
```

```
}

{
  "transaction_id": "txn_a1b2c3d4e5f6",
  "session_id": null,
  "user_id": "user_000173",
  "timestamp": "2025-04-01T18:27:15",
  "items": [
    {
      "product_id": "prod_04837",
      "quantity": 1,
      "unit_price": 59.99,
      "subtotal": 59.99
    },
    {
      "product_id": "prod_01287",
      "quantity": 3,
      "unit_price": 12.50,
      "subtotal": 37.50
    }
  ],
  "subtotal": 97.49,
  "discount": 0.00,
  "total": 97.49,
  "payment_method": "paypal",
  "status": "shipped"
}
```

Key Relationships

- **User → Sessions → Transactions**
  - A user (user_id) can have multiple sessions.
  - A session (session_id) belongs to exactly one user.
  - A session may result in zero or one transaction.
  - Transactions reference the session that generated them (when applicable). ○ *Example:* user_000042 had a session sess_a7b3c9d8e2 that resulted in transaction txn_c8d9e7f3a2b1.

- **Products → Categories → Subcategories**
    - Products are classified in a hierarchical structure.
    - Each product belongs to one category and one subcategory (as per the provided generator logic, though the sample data for products.json implies subcategory is optional or could be directly under category).
    - *Example:* prod_00123 belongs to category cat_007 and subcategory sub_007_01.
- **Sessions → Products → Cart → Transactions**
    - Sessions track viewed products in chronological order.
    - Products viewed can be added to cart during the session.
    - Cart contents may convert to transaction line items.
    - *Example:* User viewed prod_00123 during session sess_a7b3c9d8e2, added it to cart, and purchased it in transaction txn_c8d9e7f3a2b1.
- **Inventory Management Chain**
    - Products have current_stock that decreases with transactions.
    - Transactions can only be completed for in-stock products.
    - *Example:* prod_03267 has zero stock and is marked inactive.

Database-Specific Advantages Demonstrated by this Dataset

- **MongoDB (Document Model)**
    - Ideal for storing and querying:
        - Nested user session journeys with page_views array.
        - Product hierarchies with category/subcategory relationships (though the sample data for categories shows subcategories, the product sample only lists category_id and subcategory_id. The generator script links them).
        - Complex transaction documents with embedded items array.
    - ***Example Query Opportunity***: Analyzing the complete user journey from entry to purchase in a single session document, including referral source, device information, and all page views.
- **HBase (Wide-Column Store)**
    - Well-suited for:
        - Time-series session data with user_id+timestamp row keys for efficient retrieval of user activity streams.
        - Product performance tracking over time (e.g., views, additions to cart per product per day).
        - High-volume, sparse page view event storage and retrieval.
    - ***Example Query Opportunity***: Tracking product view-to-purchase conversion rates across different time periods with efficient time-range scans on user session or product interaction data.
- **Apache Spark (Distributed Processing)**
    - Enables analysis of:

        - Large-scale user behavior patterns across millions of sessions by reading and processing all session files.
        - Complex product recommendation algorithms (e.g., collaborative filtering, content-based filtering) using combined data.
        - Cross-data-source analytics, potentially joining enriched user profiles from MongoDB with detailed session logs from HBase (after loading into Spark).
    - ***Example Query Opportunity***: Building user segmentation models based on browsing patterns (from sessions), purchase history (from transactions), and demographic information (from users).

This dataset provides a comprehensive foundation for developing analytics workflows that demonstrate the complementary strengths of document, wide-column, and distributed processing approaches to big data analytics.

IMPLEMENTATION REQUIREMENTS

PART 1: Data Modeling and Storage (25%)

Your primary goal here is to design and justify appropriate schemas for MongoDB and HBase based on the types of data and the analytical queries you intend to perform.

1. **MongoDB Implementation:**
    - **Design & Justify:** Create a document model in MongoDB to store:
        - Product catalog (consider how to represent the category hierarchy and product details).
        - User profiles (consider embedding relevant, frequently accessed information like summarized purchase history or preferences).
        - Transaction documents (consider embedding line items).
    - **Implement & Query:**
        - Load a representative a significant subset of your generated data into MongoDB.
        - Implement **two** non-trivial aggregation pipelines for analytics such as:
            - Product popularity analysis (e.g., top-selling products, most viewed products).
            - Basic user segmentation (e.g., users by age group or purchasing frequency).
            - Revenue analytics (e.g., total revenue by category or over a specific period).
2. **HBase Implementation:**
    - **Design & Justify:** Design column families in HBase to efficiently store:

        - Time-series user browsing data (e.g., row key: user_id+timestamp or user_id+reverse_timestamp for recent sessions). Focus on how you would structure data for quick retrieval of a user's session activity.
        - Product performance metrics over time (e.g., row key: product_id+date).
    - **Implement & Query (Simplified Focus):**
        - Load a significant representative subset of user session data into HBase.
        - Implement queries to retrieve user session data for a specific user.
    - **Note on HBase Setup:** You can use a Dockerized HBase instance or a cloud-based HBase service if available.
3. **Deliverable:**
    - A **Design Document** (as part of your Technical Report) detailing:
        - Your data modeling decisions for both MongoDB and HBase.
        - Clear justification for your schema designs, considering query patterns, data relationships, and the strengths of each database.
        - Comparison of why certain data was placed in MongoDB vs. HBase.
    - Scripts for data loading (can be partial for full dataset) and the implemented MongoDB aggregation queries.
    - HBase schema creation commands and sample query examples (e.g., HBase shell commands or simple client code).

PART 2: Data Processing with Apache Spark (30%)

Leverage Spark for its distributed processing capabilities on the generated dataset.

1. **Batch Processing:**
    - Implement Spark batch jobs (using PySpark RDDs/DataFrames/SQL) to:
        - Clean and normalize the raw dataset (e.g., handle missing values, standardize formats from the JSON files).
        - Perform **one** of the following:
            - Calculate basic product recommendation indicators (e.g., "users who bought X also bought Y" based on transaction data, or "products frequently viewed together" based on session data).
            - Conduct a cohort analysis of user purchasing patterns (e.g., group users by registration month and analyze their spending over subsequent months).
2. **Spark SQL Analytics:**
    - Demonstrate the use of Spark SQL to:
        - Perform complex queries on the dataframes derived from your JSON files.
        - Show how Spark SQL could be used to query data that *could* notionally reside in MongoDB/HBase (e.g., by loading samples into DataFrames and joining them, or by using Spark connectors if you explore that path – though connector setup can be time-consuming). Focus on the SQL query

logic itself.

3. **Deliverable:**
    - Well-commented Spark application code (PySpark) for batch tasks.
    - Explanation of your data processing pipeline, transformations, and any optimization choices considered.
    - Spark SQL queries used.

PART 3: Analytics Integration (25%)

Showcase how these technologies can work together.

1. **Integrated Analytical Queries:**
    - Develop and implement **one** analytical query that would benefit from data potentially stored across your different systems. For example:
        - **Customer Lifetime Value (CLV) Estimation:** Combine user profile data (MongoDB) with transaction history (MongoDB) and potentially engagement metrics (HBase for session frequency/duration).
        - **Product Affinity/Recommendation:** Use transaction data (MongoDB) and browsing/view data (HBase/Spark analysis) to suggest products.
        - **Funnel Conversion Analysis:** Track user journeys from initial product view (HBase sessions) through cart addition (HBase/MongoDB) to final purchase (MongoDB transactions).
        - **Seasonal Trend Identification:** Analyze transaction data (MongoDB) possibly enriched with product view trends (HBase/Spark) over time.
    - For each query:
        - Clearly state the business question.
        - Explain which data sources (and therefore, which database/system) would be involved.
        - Describe the processing steps, highlighting where Spark might be used for joining or complex computation.
2. **Deliverable:**
    - Code (e.g., Spark jobs, Python scripts using database connectors) for the implemented integrated queries.
    - A section in your Technical Report explaining the workflow for each query, why the chosen technologies are appropriate, and any performance considerations.

PART 4: Visualization and Insights (20%)

Communicate your findings effectively.

1. **Visualizations:**
    - Create **at least 3-4 meaningful visualizations** to represent your analytical findings.

Examples:
    - Sales performance over time or by category.
    - Customer segmentation (e.g., distribution of users by age group and their average spending).
    - Product performance (e.g., top-selling vs. most-viewed products).
    - Conversion funnel stages.
  - **Tools:** You can use Python libraries like Matplotlib, Seaborn, Plotly, or any other suitable tool. The output should be static plots/images. An interactive dashboard is **not** required but can be pursued if you have the expertise and time.
2. **Deliverable:**
  - Visualizations (also embedded in your Report).

FINAL DELIVERABLES CHECKLIST

1. **GitHub Repository:**
  - All source code (Python dataset generator, MongoDB scripts, HBase schema/queries, Spark applications, integration scripts).
  - Sample queries and instructions to reproduce key results.
2. **Technical Report (PDF, target 6-8 pages):**
  - System architecture overview (how MongoDB, HBase, Spark fit together).
  - Detailed data modeling decisions and rationale (from Part 1).
  - Explanation of Spark processing pipelines (from Part 2).
  - Description of integrated analytics workflows (from Part 3).
  - Brief discussion on scalability considerations for your solution.
  - Technology selection justification for different analytical tasks.
  - Key findings and business insights (from Part 4).
  - Visualization of results.
  - Methodology explanation for key analyses.
  - Limitations and future work.

EVALUATION CRITERIA

- **Appropriate Technology Usage & Design (35%):**
  - Sound justification for data modeling choices in MongoDB and HBase.
  - Effective use of MongoDB's features for document-oriented data.
  - Clear understanding of HBase's wide-column model for time-series/sparse data (even if implementation is simplified).
  - Efficient and logical use of Spark for data processing and analytics.
  - Quality of the integration strategy for complex queries.
- **Analytics Quality & Insights (30%):**
  - Depth, relevance, and correctness of the analyses performed.
  - Appropriateness of analytical techniques chosen.

    - Clarity and actionability of business insights derived.
    - Quality of visualizations and their ability to support findings.
- **Implementation Quality (25%):**
    - Correctness, efficiency, and readability of code.
    - Logical design of data pipelines and workflows.
    - Adequate error handling and robustness (where applicable).
    - Clarity and completeness of documentation (code comments, README, reports).
- **Innovation and Critical Thinking (10%):**
    - Thoughtful approaches to analytical challenges.
    - Creative yet practical use of the technologies.
    - Insightful discussion of limitations, trade-offs, and scaling.

ADDITIONAL NOTES

- Always consider the scalability of your solution components and briefly discuss this in your technical report.

Good luck! This project is an excellent opportunity to synthesize what you've learned and build a significant piece of work.